

Proof-Carrying Possibility Atlas for Social Choice: An AI/Systems-First Sen24 Case Study

Anonymous Authors

Abstract

We present a verification-first Possibility Atlas pipeline for computational social choice. The contribution is not a new social-choice theorem, but a narrow chain of trust that turns SAT search into auditable evidence: Lean semantics and theorems on one side, DIMACS generation plus manifest auditing in the middle, and LRAT checked by the Lean kernel on the other. For the fixed Sen base case ($n = 2, m = 4$), the system enumerates SAT/UNSAT regions, extracts one MUS and one small MCS candidate for boundary explanation, and validates SAT witnesses independently of solver output strings. We state the guarantee boundary explicitly: semantic theorems and LRAT replay are proved in Lean, schema conformance is audited mechanically, and encoding correctness is claimed only up to audited sen24 invariants plus a finite check that short-cycle constraints coincide with acyclicity on four alternatives.

1 Introduction

Classical impossibility theorems explain why certain axiom sets cannot hold simultaneously, but engineering workflows also need the complementary question: *which minimal lever changes restore feasibility*. This paper frames that question as an auditable systems problem.

Our central thesis is practical: a small, proof-carrying atlas can already provide publishable insight if it fixes one clear chain of trust. In this paper, the golden thread is *auditable verification*: external SAT search is allowed to be heuristic and untrusted, but the final evidence must reduce either to Lean proofs or to replayable, independently audited artifacts. We operationalize this thesis in sen24 with claim blocks C1–C6 mapped to concrete commands and artifacts in docs/paper_claims_map.md.

Big Three Contributions.

- **Hybrid verification architecture.** We separate Lean semantics and theorem statements from SAT search, then attach LRAT certificates so that final UNSAT claims are checked by the Lean kernel rather than trusted to the solver.
- **Auditable CNF pipeline.** We pair DIMACS generation with manifests, independent auditing, clause-family invariants, and replay commands, so encoding drift is detectable instead of implicit.
- **Sen24 as dual evidence.** The fixed base case provides both a constructive Lean-side impossibility witness and a committed proof-carrying UNSAT instance, giving one semantic proof path and one exhaustive certificate path for the same small scope.

Official Claim Ladder. The paper-level claims follow the current official map exactly:

- **C1:** sen24 axiom-lever enumeration yields a reproducible SAT/UNSAT frontier.

- **C2:** UNSAT boundary cases are explainable via one MUS and one small MCS candidate.
- **C3:** UNSAT evidence can be attached as LRAT and checked in Lean.
- **C4:** symmetry reduction and monotone pruning are used only under explicit assumptions and runtime validators.
- **C5:** relaxed SAT cases yield a deterministic, auditable gallery with explicit SAT witness validation.
- **C6:** repair enumeration is checked against an independent optimum baseline by triangulation.

This ladder is intentionally conservative. It distinguishes what the repository can produce from what each revision re-verifies, and it does not broaden sen24 evidence into a claim about arbitrary n, m .

2 Problem and Positioning

We study a fixed finite domain (sen24): two voters, four alternatives, and a social welfare function represented as a binary relation. Axiom toggles are treated as levers, producing a finite family of SAT instances.

Our positioning is AI/Systems-first with COMSOC relevance:

- **AI/Systems:** reproducible pipeline design, schema versioning, solver metadata, and safety checks.
- **COMSOC:** impossibility boundary analysis, minimal contradiction causes (MUS), and minimal repair actions (MCS).

The objective is not to claim general n, m theorems, but to establish a high-integrity small-scope platform whose artifacts can be reviewed, rerun, and extended.

3 System Design

The implementation follows four separable layers. The purpose of the split is not software modularity alone; it is to keep the trusted computing base (TCB) small and explicit.

Spec. Lean definitions capture relation primitives, axioms, and theorem statements for sen24.

Encode. Python generators emit DIMACS CNF with manifests that record schema parameters, per-category clause counts, and hashes.

Solve. External SAT solvers produce SAT models or UNSAT proof traces.

Check. Trusted checks include: (i) CNF audit scripts against the manifest contract, (ii) LRAT verification in Lean for committed UNSAT references, and (iii) SAT witness validation against CNF.

This separation enables fast heuristic experimentation in Python while preserving a narrow proof-carrying core in the checking layer.

3.1 Trust Model and Chain of Trust

Figure 1 fixes what is trusted, what is not, and what kind of tampering is expected to be detectable.

Trusted semantics/checking. Lean definitions and semantic theorems; the Lean kernel and <code>FromLRAT</code>; the independent <code>sen24</code> auditor; committed artifact hashes. Untrusted search/execution. The external SAT solver, the Python encoder as an implementation artifact, the OS, and the filesystem. Chain of trust. Lean spec $\rightarrow$ audited DIMACS + manifest $\rightarrow$ untrusted SAT solving $\rightarrow$ LRAT/model outputs $\rightarrow$ Lean LRAT replay or SAT witness validation. Detection surface. CNF or manifest drift is exposed by hash mismatches and audit failure; LRAT tampering fails kernel checking; SAT model tampering fails witness validation; metadata tampering is not itself a proof failure but is auditable by replay against recorded hashes and commands.

Figure 1: The `sen24` trust model. Solvers are used for search, not as trusted decision or proof oracles.

3.2 Proof, Audit, and Assumption

We make three distinct kinds of guarantee.

Proof. The semantic `sen24` theorem is proved in Lean, and committed UNSAT references are replayed in Lean from LRAT. These statements depend on the Lean kernel rather than on the SAT solver.

Audited invariant. The DIMACS side is checked against a fixed `sen24` schema: variable numbering, clause-family membership, manifest consistency, and hash linkage are reconstructed independently of the generator. For the rationality side, we also run a finite `sen24` check showing that, under asymmetry on four alternatives, forbidding directed 3-cycles and 4-cycles is equivalent to acyclicity.

Assumption. We do not claim a fully mechanized encoding-correctness proof from Lean semantics to Python-generated CNF. The generator is trusted to implement the audited `sen24` contract, and that contract is what reviewers can inspect and replay.

3.3 Claims, Artifacts, and Commands

Table 1 is the body-level claim map extracted from `docs/paper_claims_map.md`. Its purpose is to keep the paper-level claims tied to one canonical command and one primary artifact per claim.

4 Methods

4.1 Atlas enumeration

We enumerate axiom subsets over a fixed universe and solve each case to obtain SAT/UNSAT status. Outputs include per-case CNF/manifest/summary and top-level `atlas.json`.

4.2 Boundary extraction

For UNSAT solved cases, we run a deletion-based MUS procedure and a small MCS search (single removals first, then bounded pairs). This is intentionally the narrow C2 claim: one MUS and one small MCS candidate are enough to explain the boundary case without claiming full repair enumeration.

Claim	Canonical command	Primary artifact
C1 frontier	<code>python3 scripts/run_atlas.py -outdir /tmp/atlas_c1 -jobs 4 -prune none python3 scripts/summarize_atlas.py -outdir /tmp/atlas_c1</code>	<code>/tmp/atlas_c1/atlas.json</code>
C2 one MUS + one small MCS	<code>python3 scripts/run_atlas.py -outdir /tmp/atlas_c2 -jobs 4 -prune none python3 scripts/mus_mcs.py -outdir /tmp/atlas_c2</code>	<code>/tmp/atlas_c2/case_*/mus.json, mcs.json</code>
C3 kernel-checked UNSAT	<code>python3 scripts/run_atlas.py -outdir /tmp/atlas_c3 -jobs 1 -case-masks 31 -emit-proof unsat-only lake build SocialChoiceAtlas.Sen.Atlas.Case11111</code>	<code>Certificates/atlas/case_11111/*</code>
C4 guarded acceleration	<code>python3 scripts/run_atlas.py -outdir /tmp/atlas_c4 -jobs 1 -prune monotone -prune-check -symmetry alts -symmetry-check</code>	<code>/tmp/atlas_c4/atlas.json</code>
C5 validated SAT gallery	<code>python3 scripts/run_atlas.py -outdir /tmp/atlas_c5 -jobs 4 -prune none python3 scripts/build_sat_gallery.py -atlas-outdir /tmp/atlas_c5 -top-k 5 -min-k 1</code>	<code>/tmp/atlas_c5/gallery.json</code>
C6 repair triangulation	<code>python3 scripts/run_atlas.py -outdir /tmp/atlas_c6 -jobs 4 -prune none python3 scripts/enumerate_repairs.py -outdir /tmp/atlas_c6 python3 scripts/triangulate_repairs.py -atlas-outdir /tmp/atlas_c6 -outdir /tmp/atlas_c6</code>	<code>/tmp/atlas_c6/repair_triangulation.json</code>

Table 1: Body-level claim-to-artifact-to-command map for the sen24 paper claims.

4.3 Repair enumeration and triangulation

When we want the stronger C6 evidence, we run a separate post-processing stage with `scripts/enumerate_repairs.py` and `scripts/triangulate_repairs.py`. The paper keeps this distinct from C2 so that repair *enumeration* is not confused with boundary *explanation*.

4.4 Proof-carrying UNSAT policy

For canonical UNSAT references, we retain CNF+LRAT and verify in Lean. For atlas runs, proof metadata is recorded with hash-based linkage and replay commands.

4.5 sen24 acyclicity boundary

The CNF does not encode Lean’s `Acyclic` predicate directly. Instead, for sen24 it forbids directed 3-cycles and 4-cycles. We therefore add one explicit finite check to the method: `scripts/check_acyclicity_short_cycles.py` exhaustively enumerates the four-alternative relation space and verifies that, under asymmetry, this short-cycle encoding is equivalent to acyclicity. This is a sen24-specific audited fact, not a scalability claim.

4.6 SAT Gallery with witness validation

SAT cases are filtered by lightweight non-trivial heuristics and ranked deterministically. Each selected witness is validated against CNF using a solver-independent checker that parses DIMACS

and model assignments directly.

5 Evaluation

This section is organized as a claim-by-claim evidence checklist rather than a fresh benchmark report. The goal of the current revision is to state, for each official claim C1–C6, which artifact supports it and what was actually re-verified now. The runtime harness in `scripts/eval_atlas.py` was re-run at smoke scope inside `./scripts/ci_phase2_smoke.sh`, but it is used here only as a sanity check on metadata and plotting, not as a new performance claim.

C1: reproducible frontier generation. Primary support comes from `scripts/run_atlas.py`, `atlas.json`, `atlas_summary.md`, and per-case `summary.json`. This revision re-verified C1 through `./scripts/ci_phase2_smoke.sh`, which re-runs a full 32-case sen24 atlas in the repair smoke path and checks the resulting frontier artifacts.

C2: one MUS and one small MCS candidate. Primary support comes from `scripts/mus_mcs.py` and the embedded `mus/mcs` fields in `atlas.json`. This revision re-verified the smoke-scope version of C2 through `./scripts/ci_phase2_smoke.sh`, which regenerates `mus.json` and `mcs.json` for the committed UNSAT boundary case `case_11111`.

C3: proof-carrying UNSAT. Primary support comes from the committed reference `Certificates/atlas/c`, the Lean target `SocialChoiceAtlas.Sen.Atlas.Case11111`, and proof metadata in regenerated summaries. This revision re-verified C3 through both `./scripts/ci_phase1.sh` and `./scripts/ci_phase2_smoke.sh`, which rebuild Lean proof targets and check the committed proof-carrying case.

C4: guarded symmetry reduction and monotone pruning. Primary support comes from `docs/assumptions_monotone_pruning.md`, `docs/safety_symmetry_reduction.md`, and the runtime guardrail fields in `atlas.json`. This revision re-verified C4 through `./scripts/ci_phase2_smoke.sh`, which executes both symmetry-checked and prune-checked atlas runs and validates their metadata.

C5: auditable SAT gallery. Primary support comes from `scripts/build_sat_gallery.py`, `gallery.json`, `gallery.md`, per-case `rule_card.md/rule_card.tex`, and embedded SAT witness-validation statistics. This revision re-verified C5 through `./scripts/ci_phase2_smoke.sh`, which rebuilds a small gallery, checks that selected entries are non-trivial and validated, and confirms that rule-card artifacts exist.

C6: repair triangulation against an independent optimum baseline. Primary support comes from `scripts/enumerate_repairs.py`, `scripts/triangulate_repairs.py`, `repair_triangulation.j` and the comparison fields `size_match/set_match`. This revision re-verified C6 through `./scripts/ci_phase2_s` which re-runs full repair enumeration and triangulation on a solved 32-case sen24 atlas.

6 Sen24 Case Study

The sen24 atlas is the fixed case study where the paper’s trust argument becomes concrete. It supplies two different but complementary evidence paths:

- **Semantic path.** `SocialChoiceAtlas/Sen/BaseCase24/Theorem.lean` proves the sen24 impossibility statement in Lean by constructing witness profiles that force 3-cycles or 4-cycles.

- **Artifact path.** `SocialChoiceAtlas/Sen/BaseCase24/SATSenCNF.lean` replays an LRAT certificate for the committed DIMACS baseline, proving UNSAT of the audited CNF instance in the Lean kernel.

These paths are intentionally linked but not conflated. The semantic theorem is a statement about social-choice axioms. The LRAT replay is a statement about one concrete CNF artifact. The paper’s chain of trust is the argument that these two forms of evidence are compatible under an explicit audit contract.

Acyclicity bridge for four alternatives. The main `sen24`-specific semantic gap is that the CNF uses `NO_CYCLE3/NO_CYCLE4`, while the Lean side uses `Acyclic`. We do not leave this implicit. The repository includes `scripts/check_acyclicity_short_cycles.py`, which exhaustively checks all relations on the four-alternative domain and verifies that, under asymmetry, acyclicity is equivalent to the absence of directed 3-cycles and 4-cycles. This justifies the `sen24` encoding choice without overclaiming a general theorem for arbitrary numbers of alternatives.

Figure placeholders (generated by scripts):

- `paper/figures/generated/frontier_matrix.png`
- `paper/figures/generated/frontier_boundary.png`
- `paper/figures/generated/frontier_hasse.png`

For UNSAT verification, the committed reference case uses LRAT checked by Lean. For SAT examples, the gallery records deterministic ranking and witness-validation statistics for each selected case.

Reproducible repair tables. For a fully solved atlas run (`-prune none`), axiom-level repair sets are enumerated with `scripts/enumerate_repairs.py`. The resulting `mcs_all`, `mcs_min_size`, and `mcs_min_all` fields in `atlas.json` provide inclusion-minimal SAT repairs per UNSAT case. This is C6 evidence. It is stronger than, and deliberately separate from, the narrower C2 claim of one MUS plus one small MCS candidate.

Validated non-trivial SAT rule cards. Each selected SAT gallery entry is accompanied by `rule_card.md` and `rule_card.tex`, including non-triviality metrics and concrete profile witnesses. SAT assignments are checked with `scripts/validate_sat_witness.py` before rule cards are emitted.

Triangulation and poset structure artifacts. The optimization-side triangulation report is generated as `repair_triangulation.md` via `scripts/triangulate_repairs.py`, comparing independent optimum repair sizes against `mcs_min_size/mcs_min_all`. The frontier structure is rendered as a subset-poset/Hasse graph (`frontier_hasse.dot` and `frontier_hasse.png`) via `scripts/plot_hasse_frontier.py`.

Generated tables for the paper draft. *repairs_table.tex not generated yet. gallery_table.tex not generated yet. triangulation_table.tex not generated yet.*

7 Related Work and Positioning

Our position aligns with, but does not replace, existing lines of work:

- **Constraint Hierarchies:** prioritize conflicting constraints; we focus on explicit SAT/UNSAT atlas structure.

- **ATMS**: track assumption consistency; we add proof-carrying audit artifacts and kernel-level checks.
- **OMT/MAXSAT**: optimize objective values; we prioritize boundary explanation and reproducible verification contracts.

The main differentiator is verification-first systems integration: semantics-aware encoding, manifest-audited generation, and proof-carrying checking.

8 Limitations and Scope

This paper intentionally targets a killable instance (sen24: two voters, four alternatives). We do not claim direct scalability to arbitrary n, m , and we do not claim an end-to-end mechanized encoding-correctness proof for the Python generator.

The semantic scope is also fixed:

- the domain is sen24 only;
- Universal Domain is represented by linear orders on four alternatives;
- the `NO_CYCLE3/NO_CYCLE4` encoding is justified only through the sen24 finite check under asymmetry, not as a general replacement for `Acyclic`.

Safe acceleration features (monotone pruning and symmetry reduction) are only valid under explicit assumptions and runtime guards. These assumptions are specified in:

- `docs/assumptions_monotone_pruning.md`
- `docs/safety_symmetry_reduction.md`

Future work should expand scale only if the same proof/audit/assumption split remains explicit.

9 Conclusion

We provide a practical template for proof-carrying possibility atlases in computational social choice. The sen24 study shows that small-scope exploration can still produce publishable systems evidence when the chain of trust is explicit: semantic statements are proved in Lean, concrete UNSAT artifacts are kernel-checked from LRAT, and the remaining encoder boundary is exposed as an audit contract rather than hidden inside solver trust.

The immediate next step is controlled expansion of the atlas while preserving the same trust and reproducibility guarantees.

A Reproducibility Appendix

A.1 Claim-to-artifact map

Claims C1–C6 are mapped to concrete commands and artifacts in `docs/paper_claims_map.md`.

- **C1**: atlas frontier generation.
- **C2**: one MUS and one small MCS candidate for boundary explanation.
- **C3**: proof-carrying UNSAT verification in Lean.
- **C4**: safety-guarded acceleration (symmetry/pruning).

- **C5:** SAT Gallery extraction with witness validation.
- **C6:** full repair enumeration plus triangulation against an independent optimum baseline.

Claim	Canonical command	Primary artifact
C1	python3 scripts/run_atlas.py -outdir /tmp/atlas_c1 -jobs 4 -prune none python3 scripts/summarize_atlas.py -outdir /tmp/atlas_c1	/tmp/atlas_c1/atlas.json
C2	python3 scripts/run_atlas.py -outdir /tmp/atlas_c2 -jobs 4 -prune none python3 scripts/mus_mcs.py -outdir /tmp/atlas_c2	/tmp/atlas_c2/case_*/mus.json, mcs.json
C3	python3 scripts/run_atlas.py -outdir /tmp/atlas_c3 -jobs 1 -case-masks 31 -emit-proof unsat- only lake build SocialChoiceAtlas.Sen.Atlas.Case11111	Certificates/atlas/case_11111/proof.lrat
C4	python3 scripts/run_atlas.py -outdir /tmp/atlas_c4 -jobs 1 -prune monotone -prune-check -symmetry alts -symmetry-check	/tmp/atlas_c4/atlas.json
C5	python3 scripts/run_atlas.py -outdir /tmp/atlas_c5 -jobs 4 -prune none python3 scripts/build_sat_gallery.py -atlas-outdir /tmp/atlas_c5 -top- k 5 -min-k 1	/tmp/atlas_c5/gallery.json
C6	python3 scripts/run_atlas.py -outdir /tmp/atlas_c6 -jobs 4 -prune none python3 scripts/enumerate_repairs.py -outdir /tmp/atlas_c6 python3 scripts/triangulate_repairs.py -atlas-outdir /tmp/atlas_c6 -outdir /tmp/atlas_c6	/tmp/atlas_c6/repair_triangulation.json

Table 2: Compact C1–C6 claim map with one canonical command per claim.

A.2 Verification status in this revision

In Table 3, “Repo-capable” means the repository contains the machinery and documented artifacts for the check. “Re-verified now” means the command was actually re-run in this revision.

For the sen24 acyclicity bridge used in the paper text, run: `python3 scripts/check_acyclicity_short_cycles.py -json-out results/verification/sen24_short_cycles.json`.

Check / artifact	Repo-capable	Re-verified now	Command	Expected artifact(s)
sen24 acyclicity finite check	Yes	Yes	python3 scripts/check_acyclicity_short_cycles.py -json-out results/verification/sen24_short_cycles.json	results/verification/sen24_short_cycles.json
Phase1 baseline audit + Lean replay	Yes	Yes	./scripts/ci_phase1.sh	phase1 CNF audit outputs; Lean build for SATSenCNF
Paper PDF build	Yes	Yes	make -C paper pdf	paper/build/main.pdf
Phase2 smoke harness	Yes	Yes	./scripts/ci_phase2_smoke.sh	smoke atlas, symmetry/prune checks, repair outputs, gallery outputs, bundle outputs
Tiny evidence bundle	Yes	Yes	./scripts/ci_phase2_smoke.sh	bundle.json; generated figures and paper tables in the tiny bundle
Full sen24 atlas (-prune none)	Yes	Yes	./scripts/ci_phase2_smoke.sh	32-case atlas.json used for repair and gallery smoke checks
Full repair enumeration + triangulation	Yes	Yes	./scripts/ci_phase2_smoke.sh	mcs_all; repair_triangulation.json; repair_triangulation.md
SAT gallery + rule cards	Yes	Yes	./scripts/ci_phase2_smoke.sh	gallery.json; gallery.md; per-case rule_card.md/.tex
Committed Lean proof case case_11111	Yes	Yes	./scripts/ci_phase2_smoke.sh	committed proof-carrying case plus lake build SocialChoiceAtlas.Sen.Atlas.Case11111

Table 3: Verification accounting for this revision. Status is marked “Yes” only when the command was actually re-run now.

A.3 Schema and metadata contracts

Reproducibility relies on versioned machine-readable outputs:

- atlas_schema_version
- eval_schema_version
- solver/environment metadata and seed policy fields

See docs/reproducibility_appendix.md for policy details.

A.4 Baseline protection

Protected baseline artifacts remain unchanged and must be treated as immutable references in normal development.